

## CHAPTER 1

### Lattice based cryptography

We shall require some preliminaries for introducing lattice based cryptosystems.

The dot product of  $v = (x_1, x_2, \dots, x_n), w = (y_1, y_2, \dots, y_n) \in \mathbb{R}^n$  is defined as

$$v \cdot w = x_1 y_1 + x_2 y_2 + \dots + x_n y_n.$$

The length, or Euclidean norm, of  $v$  is the quantity

$$\|v\| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}.$$

Note that  $\|v\|^2 = v \cdot v$ .

**THEOREM 1.0.1.** *Let  $v, w \in \mathbb{R}^n$ .*

- (i) *Let  $\theta$  be the angle between the vectors  $v$  and  $w$ , where we place the starting points of  $v$  and  $w$  at the origin 0. Then*

$$v \cdot w = \|v\| \|w\| \cos \theta.$$

- (ii) *(Cauchy-Schwarz inequality)*

$$|v \cdot w| \leq \|v\| \|w\|.$$

**DEFINITION 1.0.2.** Let  $S$  be a nonempty subset of a vector space  $V$ . Then the span of  $S$ , denoted by  $\text{span}(S)$ , is the collection of all finite linear combinations of elements of  $S$ . It means that

$$\text{span}(S) = \{\alpha_1 v_1 + \dots + \alpha_k v_k : \alpha_i \text{ are scalars, } v_i \in S, \ k \in \mathbb{N}\}.$$

**DEFINITION 1.0.3.** Let  $V$  be a vector space. Any subset  $S$  of  $V$  which is LI and  $\text{span}(S) = V$  is called a basis of  $V$ . The number of elements in  $S$  (provided  $S$  is a finite set) is called the dimension of  $V$ .

**REMARK 1.0.4.**

- (i) If  $B_1$  and  $B_2$  are bases of a vector space  $V$ , then there exists a bijective function from  $B_1$  to  $B_2$ . Thus if any of  $B_1$  or  $B_2$  is a finite set, then the other has to be a finite set and with the same number of elements.
- (ii) If  $\{v_1, v_2, \dots, v_k\}$  is a basis of a vector space  $V$ , then every element of  $V$  can be expressed uniquely as a linear combination of  $v_1, v_2, \dots, v_k$ .

Note that  $\{(1, 0), (0, 1)\}$  is a basis of  $\mathbb{R}^2$  and hence the dimension of  $\mathbb{R}^2$  (over  $\mathbb{R}$ ) is 2. Also, observe that the dimension of  $\mathbb{R}^n$  (over  $\mathbb{R}$ ) is  $n$ .

**DEFINITION 1.0.5.** An orthogonal basis for a vector space  $V$  is a basis  $v_1, \dots, v_n$  with the property that  $v_i \cdot v_j = 0$  for all  $i \neq j$ . The basis is orthonormal if in addition,  $\|v_i\| = 1$  for all  $i$ .

Recall that a set of  $n$  vectors  $\{v_1, v_2, \dots, v_n\}$  in  $\mathbb{R}^n$  is basis if

$$\{\alpha_1 v_1 + \alpha_2 v_2 + \dots + \alpha_n v_n : \alpha_1, \alpha_2, \dots, \alpha_n \in \mathbb{R}\} = \mathbb{R}^n.$$

Note that  $\{(1, 0), (0, 1)\}$  and  $\{(1, 3), (3, 1)\}$  are bases for  $\mathbb{R}^2$ . In fact, there are infinitely many bases for  $\mathbb{R}^n$ .

**DEFINITION 1.0.6.** Let  $B = \{v_1, v_2, \dots, v_n\}$  be a basis for  $\mathbb{R}^n$ . Then the set

$$L(B) = \{a_1 v_1 + a_2 v_2 + \dots + a_n v_n : a_1, a_2, \dots, a_n \in \mathbb{Z}\}$$

is called the lattice generated by  $B$ .

**EXAMPLE 1.0.7.** If  $\{v_1, v_2\}$  is a basis for  $\mathbb{R}^n$  and if  $w_1 = \alpha v_1 + \beta v_2$  and  $w_2 = \gamma v_1 + \delta v_2$  for some  $\alpha, \beta, \gamma, \delta \in \mathbb{Z}$  with  $\alpha\delta - \beta\gamma = \pm 1$ , then show that  $\{v_1, v_2\}$  and  $\{w_1, w_2\}$  generate the same lattice. Let  $\alpha\delta - \beta\gamma = -1$ . Let  $L_1 = \{av_1 + bv_2 : a, b \in \mathbb{Z}\}$  and  $L_2 = \{aw_1 + bw_2 : a, b \in \mathbb{Z}\}$ . Then  $L_1$  is a lattice generated by  $\{v_1, v_2\}$  and  $L_2$  is a lattice generated by  $\{w_1, w_2\}$ . We show that  $L_1 = L_2$ . Let  $x \in L_2$ . Then  $x = aw_1 + bw_2$  for some  $a, b \in \mathbb{Z}$ . Then  $x = a(\alpha v_1 + \beta v_2) + b(\gamma v_1 + \delta v_2) = (a\alpha + b\gamma)v_1 + (a\beta + b\delta)v_2 \in L_1$  as  $a\alpha + b\gamma, a\beta + b\delta \in \mathbb{Z}$ . Thus  $L_2 \subset L_1$ .

Let  $x = av_1 + bv_2 \in L_1$ . We want  $x_1, x_2 \in \mathbb{Z}$  such that  $x = x_1 w_1 + x_2 w_2 = (x_1 \alpha + x_2 \gamma)v_1 + (x_1 \beta + x_2 \delta)v_2$ . Note that  $av_1 + bv_2 = (x_1 \alpha + x_2 \gamma)v_1 + (x_1 \beta + x_2 \delta)v_2$  if and only if  $a = x_1 \alpha + x_2 \gamma$  and  $b = x_1 \beta + x_2 \delta$  if and only if  $x_1 = b\gamma - a\delta$  and  $x_2 = a\beta - b\alpha$ . Thus  $x = (b\gamma - a\delta)w_1 + (a\beta - b\alpha)w_2 \in L_2$ . This shows that  $L_1 \subset L_2$ . Hence  $L_1 = L_2$ .

The same is true in greater dimensions too. The statement is as follows.

**THEOREM 1.0.8.** Let  $B_1 = \{v_1, v_2, \dots, v_n\}$  be a basis of  $\mathbb{R}^n$ , let  $\alpha_{ij} \in \mathbb{Z}$ ,  $1 \leq i, j \leq n$ , let  $w_i = \alpha_{i1}v_1 + \alpha_{i2}v_2 + \dots + \alpha_{in}v_n$ ,  $1 \leq i \leq n$ , and let  $B_2 = \{w_1, w_2, \dots, w_n\}$ . If  $\det([\alpha_{ij}]) = \pm 1$ , then  $B_1$  and  $B_2$  generate the same lattice.

Let  $B_1 = \{v_1, v_2, \dots, v_n\}$  and  $B_2 = \{w_1, w_2, \dots, w_n\}$  be bases of  $\mathbb{R}^n$ . Then we get scalars  $\alpha_{ij}$  such that  $w_i = \sum_j \alpha_{ij} v_j$  for all  $i$ . If  $B_1$  and  $B_2$  generate the same lattice, then it is true that  $\det([\alpha_{ij}]) = \pm 1$ .

**EXERCISE 1.0.9.** Let  $B_1 = \{v_1, v_2, v_3\}$  be a basis of  $\mathbb{R}^3$ , and let  $B_2 = \{w_1, w_2, w_3\}$ , where  $w_1 = v_1 + v_3$ ,  $w_2 = v_1 - v_2 + 2v_3$ , and  $w_3 = v_1 + 2v_2$ . Show that  $B_1$  and  $B_2$  generate the same lattice.

**EXERCISE 1.0.10.** Let  $L$  be the lattice given by the basis  $B = \{(3, 1, -2), (1, -3, 5), (4, 2, 1)\}$ . Which of the following sets of vectors are also bases for  $L$ ? (a)  $B_1 = \{(5, 13, -13), (0, -4, 2), (-7, -13, 18)\}$ . (b)  $B_2 = \{(4, -2, 3), (6, 6, -6), (-2, -4, 7)\}$ .

## 1. The Shortest Vector Problem

We begin with a description of two fundamental lattice problems.

**The Shortest Vector Problem (SVP):** Find a shortest nonzero vector in a lattice  $L$ , i.e., find a nonzero vector  $v \in L$  that minimizes the Euclidean norm  $\|v\|$ .

**1.1. Two-Dimensional Lattices Lattice Reduction.** Let  $\{v_1, v_2\}$  form the basis of a two-dimensional lattice. Our first goal is to replace this basis with what will be called a reduced basis. If  $\|v_1\| > \|v_2\|$ , then swap  $v_1$  and  $v_2$ , so we may assume that  $\|v_1\| \leq \|v_2\|$ . Ideally, we would like to replace  $v_2$  with a vector  $v_2^*$  perpendicular to  $v_1$ . As in the Gram-Schmidt process from linear algebra, the vector

$$v_2^* = v_2 - \frac{v_1 \cdot v_2}{v_1 \cdot v_1} v_1$$

is perpendicular to  $v_1$ . But this vector might not lie in the lattice. Instead, let  $t$  be the closest integer to  $(v_1 \cdot v_2)/(v_1 \cdot v_1)$  (for definiteness, take 0 to be the closest integer to  $\pm \frac{1}{2}$ , and  $\pm 1$  to be closest to  $\pm \frac{3}{2}$ , etc.). Then we replace the basis  $\{v_1, v_2\}$  with the basis  $\{v_1, v_2 - tv_1\}$ . We then repeat the process with this new basis.

**DEFINITION 1.1.1.** A basis  $\{v_1, v_2\}$  is called reduced if  $\|v_1\| \leq \|v_2\|$  and  $-\frac{1}{2} \leq \frac{v_1 \cdot v_2}{v_1 \cdot v_1} \leq \frac{1}{2}$ .

The above reduction process stops exactly when we obtain a reduced basis. Observe that the reduced basis obtained from a basis generate the same lattice.

**EXAMPLE 1.1.2.** Find a reduced basis from the basis  $\{(31, 59), (37, 70)\}$ .

We have  $v_1 = (31, 59)$  and  $v_2 = (37, 70)$ . We have  $\|v_1\| < \|v_2\|$ , so we do not swap the two vectors. Since  $\frac{v_1 \cdot v_2}{v_1 \cdot v_1} = \frac{5277}{4442}$ , we take  $t = 1$ . The new basis is  $v'_1 = v_1 = (31, 59)$  and  $v'_2 = v_2 - v_1 = (6, 11)$ . Swap  $v'_1$  and  $v'_2$  and rename the vectors to obtain a basis  $v''_1 = (6, 11)$  and  $v''_2 = (31, 59)$ . We have  $\frac{v''_1 \cdot v''_2}{v''_1 \cdot v''_1} = \frac{835}{157}$ , so we take  $t = 5$ . This yields vectors  $(6, 11)$  and  $(1, 4) = (31, 59) - 5(6, 11)$ . Swap these and name them  $v^{(3)}_1 = (1, 4)$  and  $v^{(3)}_2 = (6, 11)$ . We have  $\frac{v^{(3)}_1 \cdot v^{(3)}_2}{v^{(3)}_1 \cdot v^{(3)}_1} = \frac{50}{17}$ , so  $t = 3$ . This yields, after a swap,  $v^r_1 = (3, -1)$  and  $v^r_2 = (1, 4)$ . Since  $\|v^r_1\| < \|v^r_2\|$  and  $\frac{v^r_1 \cdot v^r_2}{v^r_1 \cdot v^r_1} = -\frac{1}{10}$ , the basis  $\{v^r_1, v^r_2\}$  is reduced.

A natural question is whether this process always produces a reduced basis. The answer is yes. Moreover, the first vector in the reduced basis is a shortest vector for the lattice.

**EXAMPLE 1.1.3.** If  $\{v_1, v_2\}$  is a reduced basis for a lattice  $L$ , then show that  $\|v_1\| \leq \|v\|$  for all  $v \in L$ . It means that  $v_1$  is a shortest vector from the lattice  $L$ .

Since  $\{v_1, v_2\}$  is a reduced basis, we have  $\|v_1\| \leq \|v_2\|$  and  $-\frac{1}{2} \leq \frac{v_1 \cdot v_2}{v_1 \cdot v_1} \leq \frac{1}{2}$  or  $\left| \frac{v_1 \cdot v_2}{v_1 \cdot v_1} \right| = \left| \frac{v_1 \cdot v_2}{\|v_1\|^2} \right| \leq \frac{1}{2}$ . Let  $a$  and  $b$  be integers. Then  $|2abv_1 \cdot v_2| = |ab||2v_1 \cdot v_2| \leq |ab|\|v_1\|^2$  or  $-|ab|\|v_1\|^2 \leq 2ab(v_1 \cdot v_2) \leq |ab|\|v_1\|^2$ . We also note that  $|v_1 \cdot v_2| \leq \|v_1\|\|v_2\|$  (it is called the Cauchy-Schwarz inequality). Let  $v \in L$  be nonzero. Then we get two integers  $a$  and  $b$  such that  $v = av_1 + bv_2$ . Now,

$$\|v\|^2 = (av_1 + bv_2) \cdot (av_1 + bv_2) = a^2(v_1 \cdot v_1) + 2ab(v_1 \cdot v_2) + b^2(v_2 \cdot v_2)$$

$$\begin{aligned}
&= a^2\|v_1\|^2 + 2ab(v_1 \cdot v_2) + b^2\|v_2\|^2 \\
&\geq a^2\|v_1\|^2 - |ab|\|v_1\|^2 + b^2\|v_2\|^2 \quad (\because \text{above inequality}) \\
&\geq a^2\|v_1\|^2 - |ab|\|v_1\|^2 + b^2\|v_1\|^2 \quad (\because \|v_1\| \leq \|v_2\|) \\
&\geq |a|^2\|v_1\|^2 - |ab|\|v_1\|^2 + |b|^2\|v_1\|^2 \\
&\geq (|a|^2 - |ab| + |b|^2)\|v_1\|^2.
\end{aligned}$$

Since  $a$  and  $b$  are integers,  $|a|^2 - |ab| + |b|^2$  is an integer. Also  $|a|^2 - |ab| + |b|^2 = (|a| - |b|)^2 + |ab|$ . Thus  $|a|^2 - |ab| + |b|^2$  is a nonnegative integer. Since  $v \neq 0$ ,  $a \neq 0$  or  $b \neq 0$  and hence  $|a|^2 - |ab| + |b|^2$  is a positive integer. Thus  $|a|^2 - |ab| + |b|^2 \geq 1$ . Thus  $\|v\|^2 \geq (|a|^2 - |ab| + |b|^2)\|v_1\|^2 \geq \|v_1\|^2$  or  $\|v\| \geq \|v_1\|$ .

**EXAMPLE 1.1.4.** Find a shortest (nonzero) vector in the lattice  $L$  generated by  $\{(31, 59), (37, 70)\}$ . We found (earlier) a reduced basis  $\{(3, -1), (1, 4)\}$  from the basis  $\{(31, 59), (37, 70)\}$ . Both these bases generate  $L$ . By above example  $(3, -1)$  is a shortest (nonzero) vector in  $L$ .

## 2. The Closest Vector Problem and Babai's Algorithm

**The Closest Vector Problem (CVP):** Given a vector  $w \in \mathbb{R}^n$  that is not in  $L$ , find a vector  $v \in L$  that is closest to  $w$ , i.e., find a vector  $v \in L$  that minimizes the Euclidean norm  $\|w - v\|$ .

Both SVP and CVP are profound problems, and both become computationally difficult as the dimension  $n$  of the lattice grows. On the other hand, even approximate solutions to SVP and CVP turn out to have surprisingly many applications in different fields of pure and applied mathematics. In full generality, CVP is known to be NP-hard and SVP is NP-hard under a certain “randomized reduction hypothesis”.

**THEOREM 1.2.1 (Hermite's Theorem).** *Every lattice  $L$  of dimension  $n$  contains a nonzero vector  $v \in L$  satisfying*

$$\|v\| \leq \sqrt{n} \det(L)^{1/n}.$$

**THEOREM 1.2.2.** *Let  $L \subset \mathbb{R}^n$  be a lattice. If  $\{v_1, v_2, \dots, v_n\}$  and  $\{w_1, w_2, \dots, w_n\}$  are bases of  $L$ , then  $|\det(v_1, v_2, \dots, v_n)| = |\det(w_1, w_2, \dots, w_n)|$ .*

Thus for any basis  $\{v_1, v_2, \dots, v_n\}$  of a lattice  $L$ , we get the same number  $|\det(v_1, v_2, \dots, v_n)|$ . This number will be denoted by  $\det(L)$ .

**DEFINITION 1.2.3.** The Hadamard ratio of the basis  $B = \{v_1, \dots, v_n\}$  is defined to be the quantity

$$H(B) = \left( \frac{\det L}{\|v_1\| \|v_2\| \cdots \|v_n\|} \right)^{\frac{1}{n}}.$$

Thus  $0 < H(B) \leq 1$ , and the closer that the value is to 1, the more orthogonal are the vectors in the basis. (The reciprocal of the Hadamard ratio is sometimes called the orthogonality defect.) A basis  $B = \{v_1, v_2, \dots, v_n\}$  is a “good basis” if  $H(B)$  is close to 1.

**THEOREM 1.2.4 (Babai's Closest Vertex Algorithm).** *Let  $L \subset \mathbb{R}^n$  be a lattice with basis  $v_1, \dots, v_n$ , and let  $w \in \mathbb{R}^n$  be an arbitrary vector. If the vectors in the basis are sufficiently orthogonal to one another, then the following algorithm solves CVP.*

*Write  $w = t_1v_1 + t_2v_2 + \dots + t_nv_n$  with  $t_1, \dots, t_n \in \mathbb{R}$ .*

*Set  $a_i = [t_i]$  for  $i = 1, 2, \dots, n$ , where  $a_i = [t_i]$  is the nearest integer to  $t_i$ .*

*Return the vector  $v = a_1v_1 + a_2v_2 + \dots + a_nv_n$ .*

In general, if the vectors in the basis are reasonably orthogonal to one another, then the algorithm solves some version of apprCVP, but if the basis vectors are highly nonorthogonal, then the vector returned by the algorithm is generally far from the lattice vector that is closest to  $w$ .

**EXAMPLE 1.2.5.** Let  $L \subset \mathbb{R}^2$  be the lattice generated by the basis  $v_1 = (137, 312)$  and  $v_2 = (215, -187)$ . We use Babai's algorithm to find a vector in  $L$  that is close to the vector  $w = (53172, 81743)$ .

First we need to find  $t_1, t_2 \in \mathbb{R}$  such that  $w = t_1v_1 + t_2v_2$ , i.e.,  $(53172, 81743) = t_1(137, 312) + t_2(215, -187)$ . Solving the two equations  $53172 = 137t_1 + 215t_2$  and  $81743 = 312t_1 - 187t_2$ , we get  $t_1 \approx 296.85$  and  $t_2 \approx 58.15$ . Take  $a_1 = 297$  and  $t_2 = 58$ . Then  $v = a_1v_1 + a_2v_2 = (53159, 81818)$  is an approximate vector close to  $w = (53172, 81743)$ .

**EXAMPLE 1.2.6.** Let  $L \subset \mathbb{R}^2$  be the lattice given by the basis  $v_1 = (213, -437)$  and  $v_2 = (312, 105)$ , and let  $w = (43127, 11349)$ .

- (i) Use Babai's algorithm to find a vector  $v \in L$  that is close to  $w$ . Compute the distance  $\|v - w\|$ .
- (ii) What is the value of the Hadamard ratio  $\left(\frac{\det(L)}{\|v_1\|\|v_2\|}\right)^{\frac{1}{2}}$ ? Is the basis  $\{v_1, v_2\}$  a good basis?
- (iii) Show that the vectors  $v'_1 = (2937, -1555)$  and  $v'_2 = (11223, -5888)$  are also a basis for  $L$  by expressing them as linear combinations of  $v_1$  and  $v_2$  and checking that the change-of-basis matrix has integer coefficients and determinant 1.
- (iv) Use Babai's algorithm with the basis  $\{v'_1, v'_2\}$  to find a vector  $v \in L$ . Compute the distance  $\|v - w\|$  and compare it to your answer from (i).
- (v) Compute the Hadamard ratio using  $v'_1$  and  $v'_2$ . Is  $\{v'_1, v'_2\}$  a good basis?

Let

$$v_1 = (213, -437), \quad v_2 = (312, 105), \quad w = (43127, 11349).$$

We write  $w$  in the basis  $\{v_1, v_2\}$  by solving

$$c_1v_1 + c_2v_2 = w.$$

Thus

$$\begin{pmatrix} 213 & 312 \\ -437 & 105 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \end{pmatrix} = \begin{pmatrix} 43127 \\ 11349 \end{pmatrix}.$$

The determinant is

$$\det = 213 \cdot 105 - 312(-437) = 158709.$$

Using the inverse matrix we obtain

$$c_1 \approx 6.2217, \quad c_2 \approx 133.9800.$$

Rounding gives

$$(c_1, c_2) = (6, 134).$$

Hence the lattice vector is

$$v = 6v_1 + 134v_2.$$

Compute

$$6v_1 = (1278, -2622), \quad 134v_2 = (41808, 14070).$$

Therefore

$$v = (43086, 11448).$$

Distance:

$$v - w = (-41, 99).$$

Hence

$$\|v - w\| = \sqrt{(-41)^2 + 99^2} = \sqrt{11482} \approx 107.15.$$

The lattice determinant is

$$\det(L) = 158709.$$

Norms of the basis vectors:

$$\|v_1\| = \sqrt{213^2 + 437^2} \approx 486.15,$$

$$\|v_2\| = \sqrt{312^2 + 105^2} \approx 329.19.$$

Thus the Hadamard ratio is

$$\left( \frac{\det(L)}{\|v_1\| \|v_2\|} \right)^{1/2} = \left( \frac{158709}{486.15 \times 329.19} \right)^{1/2} \approx 0.996.$$

Hence  $\{v_1, v_2\}$  is a very good basis.

Let

$$v'_1 = (2937, -1555), \quad v'_2 = (11223, -5888).$$

Solving for linear combinations gives

$$v'_1 = 5v_1 + 6v_2,$$

$$v'_2 = 19v_1 + 23v_2.$$

Thus the change of basis matrix is

$$U = \begin{pmatrix} 5 & 19 \\ 6 & 23 \end{pmatrix}.$$

Its determinant is

$$\det(U) = 5 \cdot 23 - 19 \cdot 6 = 115 - 114 = 1.$$

Since the matrix has integer entries and determinant 1, the vectors  $v'_1, v'_2$  form another basis for the lattice  $L$ .

Solve

$$c_1 v'_1 + c_2 v'_2 = w.$$

This gives approximately

$$c_1 \approx -2402.52, \quad c_2 \approx 632.57.$$

Rounding gives

$$(c_1, c_2) = (-2403, 633).$$

Hence

$$v = -2403v'_1 + 633v'_2.$$

This yields approximately

$$v = (46548, 9561).$$

Thus

$$\|v - w\| \approx 3860.08.$$

This is much worse than the result obtained using the basis  $\{v_1, v_2\}$ .

$$\|v'_1\| = \sqrt{2937^2 + 1555^2} \approx 3323.25,$$

$$\|v'_2\| = \sqrt{11223^2 + 5888^2} \approx 12673.76.$$

Thus

$$\left( \frac{\det(L)}{\|v'_1\| \|v'_2\|} \right)^{1/2} = \left( \frac{158709}{3323.25 \times 12673.76} \right)^{1/2} \approx 0.0614.$$

Hence  $\{v'_1, v'_2\}$  is a very poor basis.

### 3. The GGH Public Key Cryptosystem

Alice begins by choosing a set of linearly independent vectors

$$v_1, v_2, \dots, v_n \in \mathbb{Z}^n$$

that are reasonably orthogonal to one another. One way to do this is to fix a parameter  $d$  and choose the coordinates of  $v_1, \dots, v_n$  randomly between  $-d$  and  $d$ . Alice can check that her choice of vectors is good by computing the Hadamard ratio of her basis and verifying that it is not too small. The vectors  $v_1, \dots, v_n$  are Alice's private key. For convenience, we let  $V$  be the  $n \times n$  matrix whose rows are the vectors  $v_1, \dots, v_n$ , and we let  $L$  be the lattice generated by these vectors.

Alice next chooses an  $n \times n$  matrix  $U$  with integer coefficients and  $\det(U) = \pm 1$ . One way to create  $U$  is as a product of a large number of randomly chosen elementary matrices. She then computes

$$W = UV.$$

The row vectors  $w_1, \dots, w_n$  of  $W$  are a new basis for  $L$ . They are Alice's public key.

When Bob wants to send a message to Alice, he selects a small vector  $m$  with integer coordinates as his plaintext, e.g.,  $m$  might be a binary vector. Bob also chooses a small random perturbation vector  $r$  that acts as a random element. For example, he might choose the coordinates of  $r$  randomly between  $-\delta$  and  $\delta$ , where  $\delta$  is a fixed public parameter. He then computes the vector

$$c = mW + r = \sum_{i=1}^n m_i w_i + r,$$

which is his ciphertext. Notice that  $c$  is not a lattice point, but it is close to the lattice point  $mW$ , since  $r$  is small.

Decryption is straightforward. Alice uses Babai's algorithm with the good basis  $v_1, \dots, v_n$  to find a vector in  $L$  that is close to  $c$ . Since she is using a good basis and  $r$  is small, the lattice vector that she finds is  $mW$ . She then multiplies by  $W^{-1}$  to recover  $m$ . The GGH cryptosystem is summarized in the following table.

| Alice                                                                                                                                                                                                                       | Bob                                                                                                                                                                                        |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Key Creation</b>                                                                                                                                                                                                         |                                                                                                                                                                                            |
| Choose a good basis $v_1, \dots, v_n$<br>Choose an integer matrix $U$<br>satisfying $\det(U) = \pm 1$ .<br>Compute a bad basis $w_1, \dots, w_n$<br>as the rows of $W = UV$ .<br>Publish the public key $w_1, \dots, w_n$ . |                                                                                                                                                                                            |
| <b>Encryption</b>                                                                                                                                                                                                           |                                                                                                                                                                                            |
|                                                                                                                                                                                                                             | Choose small plaintext vector $m$ .<br>Choose random small vector $r$ .<br>Use Alice's public key to compute<br>$c = m_1 w_1 + \dots + m_n w_n + r$ .<br>Send the ciphertext $c$ to Alice. |
| <b>Decryption</b>                                                                                                                                                                                                           |                                                                                                                                                                                            |
| Use Babai's algorithm to compute<br>the vector $v \in L$ closest to $c$ .<br>Compute $vW^{-1}$ to recover $m$ .                                                                                                             |                                                                                                                                                                                            |

EXAMPLE 1.3.1. Let  $L \subset \mathbb{R}^2$  be the lattice generated by a basis  $B = \{(17, 0), (0, 19)\}$  and the associated matrix will be  $V = \begin{pmatrix} 17 & 0 \\ 0 & 19 \end{pmatrix}$ . Let  $U = \begin{pmatrix} 2 & 3 \\ 3 & 5 \end{pmatrix}$ . Then  $W = UV = \begin{pmatrix} 2 & 3 \\ 3 & 5 \end{pmatrix} \begin{pmatrix} 17 & 0 \\ 0 & 19 \end{pmatrix} = \begin{pmatrix} 34 & 57 \\ 51 & 95 \end{pmatrix}$ . Let the message be  $m = (2, -5)$  and take  $r = (1, -1)$ . Then  $c = mW + r = (2, -5) \begin{pmatrix} 34 & 57 \\ 51 & 95 \end{pmatrix} + (1, -1) = (-186, -362)$ . To decrypt one does the following computation:  $(-186, -362) = -\frac{186}{17}(17, 0) + -\frac{362}{19}(0, 19)$ . Now,  $[-\frac{186}{17}] = -11$ . Rounding of  $-\frac{362}{19}, -\frac{186}{17}$ , we have  $-\frac{362}{19} \approx -19$  and  $-\frac{186}{17} \approx -11$ . So,  $v = -11(17, 0) + (-19)(0, 19) = (-187, -360)$ . Also, note that  $W^{-1} = \frac{1}{323} \begin{pmatrix} 95 & -57 \\ -51 & 34 \end{pmatrix}$ . Now, to recover  $m$  we compute

$$vW^{-1} = \frac{1}{323}(-187, -360) \begin{pmatrix} 95 & -57 \\ -51 & 34 \end{pmatrix} = (2, -5) = m.$$

EXAMPLE 1.3.2. Alice uses the GGH cryptosystem with private basis  $v_1 = (4, 13)$ ,  $v_2 = (-57, -45)$ , and public basis  $w_1 = (25453, 9091)$ ,  $w_2 = (-16096, -5749)$ .

- (i) Compute the determinant of Alice's lattice and the Hadamard ratio of the private and public bases.
- (ii) Bob sends Alice the encrypted message  $e = (155340, 55483)$ . Use Alice's private basis to decrypt the message and recover the plaintext. Also determine Bob's random perturbation  $r$ .
- (iii) Try to decrypt Bob's message using Babai's algorithm with the public basis  $\{w_1, w_2\}$ . Is the output equal to the plaintext?

Let  $B = \{v_1, v_2\}$  and  $B' = \{w_1, w_2\}$ .

(i) **Determinant and Hadamard ratios**

Private basis:

$$v_1 = (4, 13), \quad v_2 = (-57, -45)$$

$$\det(L) = \left\| \begin{pmatrix} 4 & -57 \\ 13 & -45 \end{pmatrix} \right\| = |4(-45) - 13(-57)| = |-180 + 741| = 561.$$

Hence

$$\det(L) = 561.$$

For the private basis,

$$\|v_1\| = \sqrt{4^2 + 13^2} = \sqrt{185}, \quad \|v_2\| = \sqrt{(-57)^2 + (-45)^2} = \sqrt{5274}.$$

Thus

$$H(B) = \frac{561}{\sqrt{185}\sqrt{5274}} \approx 0.568.$$

Public basis:

$$w_1 = (25453, 9091), \quad w_2 = (-16096, -5749).$$

$$\|w_1\| = \sqrt{25453^2 + 9091^2}, \quad \|w_2\| = \sqrt{16096^2 + 5749^2}.$$

$$H(B') = \frac{561}{\|w_1\|\|w_2\|} \approx 1.21 \times 10^{-6}.$$

**(ii) Decryption using the private basis**

Ciphertext:

$$e = (155340, 55483).$$

Let

$$B = \begin{pmatrix} 4 & -57 \\ 13 & -45 \end{pmatrix}.$$

Compute

$$B^{-1}e \approx (-6823.12, -3204.08).$$

Rounding to the nearest integers gives

$$(-6823, -3204).$$

Hence the closest lattice vector is

$$m = -6823v_1 - 3204v_2.$$

$$m = (155336, 55481).$$

Thus the plaintext lattice point is

$$m = (155336, 55481).$$

The perturbation is

$$r = e - m.$$

$$r = (155340, 55483) - (155336, 55481) = (4, 2).$$

(iii) **Babai algorithm using the public basis**

Using the public basis matrix

$$W = \begin{pmatrix} 25453 & -16096 \\ 9091 & -5749 \end{pmatrix}.$$

Compute

$$W^{-1}e \approx (-8.39, -22.92).$$

Rounding gives

$$(-8, -23).$$

The corresponding lattice vector is

$$-8w_1 - 23w_2 = (166584, 59499).$$

Since

$$(166584, 59499) \neq (155336, 55481),$$

Babai's algorithm with the public basis does *not* recover the plaintext.

**EXAMPLE 1.3.3.** Alice uses the GGH cryptosystem with private basis  $v_1 = (58, 53, -68)$ ,  $v_2 = (-110, -112, 35)$ ,  $v_3 = (-10, -119, 123)$  and public basis  $w_1 = (324850, -1625176, 2734951)$ ,  $w_2 = (165782, -829409, 1395775)$ ,  $w_3 = (485054, -2426708, 4083804)$ .

- (i) Compute the determinant of Alice's lattice and the Hadamard ratio of the private and public bases.
- (ii) Bob sends Alice the encrypted message  $c = (8930810, -44681748, 75192665)$ . Use Alice's private basis to decrypt the message and recover the plaintext. Also determine Bob's random perturbation  $r$ .
- (iii) Try to decrypt Bob's message using Babai's algorithm with the public basis  $\{w_1, w_2, w_3\}$ . Is the output equal to the plaintext?

Alice uses the GGH cryptosystem with private basis

$$v_1 = (58, 53, -68), \quad v_2 = (-110, -112, 35), \quad v_3 = (-10, -119, 123)$$

and public basis  $w_1 = (324850, -1625176, 2734951)$ ,  $w_2 = (165782, -829409, 1395775)$ ,  $w_3 = (485054, -2426708, 4083804)$ .

- (i) Compute the determinant of Alice's lattice and the Hadamard ratio of the private and public bases.

The matrix of the private basis is

$$V = \begin{pmatrix} 58 & -110 & -10 \\ 53 & -112 & -119 \\ -68 & 35 & 123 \end{pmatrix}.$$

The determinant is

$$\det(V) = 672858.$$

Hence the determinant of Alice's lattice is

$$\det(L) = 672858.$$

The Hadamard ratio of a basis  $\{b_1, b_2, b_3\}$  is

$$HR(B) = \frac{|\det(B)|}{\|b_1\| \|b_2\| \|b_3\|}.$$

First compute the norms:

$$\|v_1\| = \sqrt{58^2 + 53^2 + (-68)^2} = \sqrt{10797},$$

$$\|v_2\| = \sqrt{(-110)^2 + (-112)^2 + 35^2} = \sqrt{25869},$$

$$\|v_3\| = \sqrt{(-10)^2 + (-119)^2 + 123^2} = \sqrt{29470}.$$

Thus the Hadamard ratio of the private basis is

$$HR_{private} = \frac{672858}{\sqrt{10797}\sqrt{25869}\sqrt{29470}} \approx 0.235.$$

For the public basis, the determinant is the same:

$$\det(W) = 672858.$$

Hence

$$HR_{public} = \frac{672858}{\|w_1\| \|w_2\| \|w_3\|} \approx 2.7 \times 10^{-14}.$$

Thus the public basis is extremely bad compared to the private basis.

(ii) Bob sends Alice the encrypted message

$$c = (8930810, -44681748, 75192665).$$

To decrypt, compute the approximate coordinates of  $c$  in the private basis:

$$V^{-1}c \approx (-334865.226, -304373.024, 512803.947).$$

Rounding to the nearest integers gives

$$x = (-334865, -304373, 512804).$$

The closest lattice vector is

$$m = Vx.$$

This gives

$$m = (8930820, -44681745, 75192657).$$

Thus the recovered plaintext lattice vector is

$$m = (8930820, -44681745, 75192657).$$

The perturbation vector is

$$r = c - m.$$

Hence

$$r = (-10, -3, 8).$$

(iii) Now apply Babai's algorithm using the public basis  $\{w_1, w_2, w_3\}$ .

Compute

$$W^{-1}c \approx (51.588, 416.672, -158.548).$$

Rounding gives

$$(52, 417, -159).$$

The resulting lattice vector is

$$W(52, 417, -159) = (8899708, -44526133, 74930791).$$

This vector is not equal to the plaintext lattice vector

$$(8930820, -44681745, 75192657).$$

Hence Babai's algorithm with the public basis does not recover the plaintext.

**EXAMPLE 1.3.4.** We illustrate the GGH cryptosystem with a 3-dimensional example. For Alice's private good basis we take  $v_1 = (-97, 19, 19)$ ,  $v_2 = (-36, 30, 86)$ ,  $v_3 = (-184, -64, 78)$ . The lattice  $L$  spanned by  $v_1, v_2$ , and  $v_3$  has determinant  $\det(L) = 859516$ , and the Hadamard ratio of the basis is

$$H(v_1, v_2, v_3) = \left( \frac{\det(L)}{\|v_1\| \|v_2\| \|v_3\|} \right)^{\frac{1}{3}} \approx 0.74620.$$

Alice multiplies her private basis by the matrix

$$U = \begin{pmatrix} 4327 & -15447 & 23454 \\ 3297 & -11770 & 17871 \\ 5464 & -19506 & 29617 \end{pmatrix},$$

which has determinant  $\det(U) = -1$ , to create her public basis  $w_1 = (-4179163, -1882253, 583183)$ ,  $w_2 = (-3184353, -1434201, 444361)$ ,  $w_3 = (-5277320, -2376852, 736426)$ . The Hadamard ratio of the public basis is very small,

$$H(w_1, w_2, w_3) = \left( \frac{\det(L)}{\|w_1\| \|w_2\| \|w_3\|} \right)^{\frac{1}{3}} \approx 0.0000208.$$

Bob decides to send Alice the plaintext  $m = (86, -35, -32)$  using the random element  $r = (-4, -3, 2)$ . The corresponding ciphertext is

$$\begin{aligned} c &= (86, -35, -32) \begin{pmatrix} -4179163 & -1882253 & 583183 \\ -3184353 & -1434201 & 444361 \\ -5277320 & -2376852 & 736426 \end{pmatrix} + (-4, -3, 2) \\ &= (-79081427, -35617462, 11035473). \end{aligned}$$

Alice uses Babai's algorithm to decrypt. She first writes  $c$  as a linear combination of her private basis with real coefficients,

$$c \approx 81878.97v_1 - 292300.00v_2 + 443815.04v_3.$$

She rounds the coefficients to the nearest integer and computes a lattice vector

$$v = 81879v_1 - 292300v_2 + 443815v_3 = (-79081423, -35617459, 11035471)$$

that is close to  $c$ . She then recovers  $m$  by expressing  $v$  as a linear combination of the public basis and reading off the coefficients,

$$v = 86w_1 - 35w_2 - 32w_3.$$

Now suppose that Eve tries to decrypt Bob's message, but she knows only the public basis  $w_1, w_2, w_3$ . If she applies Babai's algorithm using the public basis, she finds that  $c \approx 75.76w_1 - 34.52w_2 - 24.18w_3$ . Rounding, she obtains  $v' = 76w_1 - 35w_2 - 24w_3 = (-79508353, -35809745, 11095049)$  (a lattice vector) that is somewhat close to  $c$ . However, this lattice vector gives the incorrect plaintext  $(76, -35, -24)$ , not the correct plaintext  $m = (86, -35, -32)$ . It is instructive to compare how well Babai's algorithm did for the different bases. We find that

$$\|e - v\| \approx 5.39 \text{ and } \|e - v'\| \approx 472004.09.$$

Of course, the GGH cryptosystem is not secure in dimension 3, since even if we use numbers that are large enough to make an exhaustive search impractical, there are efficient algorithms to find good bases in low dimension. In dimension 2, an algorithm for finding a good basis dates back to Gauss. A powerful generalization to arbitrary dimension, known as the LLL algorithm.

## CHAPTER 2

### Quantum Key Distribution

#### 1. BB84

Below we describe the famous BB84 quantum key distribution (QKD) protocol of Bennett and Brassard. Consider two possible bases: basis 0 is the computational basis  $\{|0\rangle, |1\rangle\}$ , and basis 1 is the Hadamard basis  $\{|+\rangle, |-\rangle\}$ . The main property of quantum mechanics that we'll use, is that if a bit  $b$  is encoded in an unknown basis, then Eve cannot get information about  $b$  without disturbing the state, and the latter can be detected by Alice and Bob.

- (i) Alice chooses  $n$  random bits  $a_1, \dots, a_n$  and  $n$  random bases  $b_1, \dots, b_n$ . She sends  $a_i$  to Bob in basis  $b_i$  over the public quantum channel. For example, if  $a_i = 0$  and  $b_i = 1$ , then the  $i$ -th qubit that she sends is in state  $|+\rangle$ .
- (ii) Bob chooses random bases  $b'_1, \dots, b'_n$  and measures the qubits he received in those bases, yielding bits  $a'_0, \dots, a'_n$ .
- (iii) Bob sends Alice all  $b'_i$  (this also signals to Alice that Bob has measured the qubits he received), and Alice sends Bob all  $b_i$ . Note that for roughly  $n/2$  of the  $i$ s, Alice and Bob used the same basis  $b_i = b'_i$ . For those  $i$  Bob should have  $a'_i = a_i$  (if there was no noise and Eve didn't tamper with the  $i$ -th qubit on the channel). Both Alice and Bob know for which  $i$  this holds. Let's call these roughly  $n/2$  positions the "shared string."
- (iv) Alice randomly selects  $n/4$  locations in the shared string, and sends Bob those locations as well as the values  $a_i$  at those locations. Bob then checks whether they have the same bits in those positions. If the fraction of errors is bigger than some number  $p$ , then they suspect some eavesdropper was tampering with the channel, and they abort.
- (v) If the test is passed, then they discard the  $n/4$  test-bits, and have roughly  $n/4$  bits left in their shared string. This is called the "raw key." Now they do some classical postprocessing on the raw key: "information reconciliation" to ensure they end up with exactly the same shared string, and "privacy amplification" to ensure that Eve has negligible information about that shared string.

**EXAMPLE 2.1.1.** Suppose that Alice chooses a string  $a = 1010010001$ ,  $b = 1001011010$  and Bob chooses  $b' = 0100100011$ . What is the shared key?

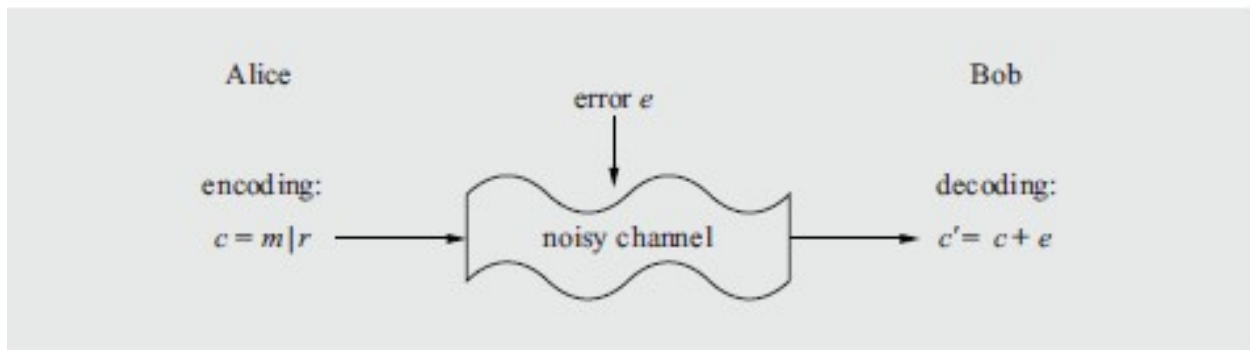
**EXERCISES 2.1.2.**

- (i) Given the complex number  $c = 3 - 4i$ , calculate its complex conjugate  $c^*$  and its absolute square  $|c|^2$ . Find a real scaling factor  $A$  such that  $|A \cdot c|^2 = 1$ .
- (ii) Convert the complex numbers  $z_1 = 1 + i$  and  $z_2 = -\sqrt{3} + i$  into their polar forms  $re^{i\theta}$ .
- (iii) Let  $|v\rangle = \begin{pmatrix} 1+i \\ 2 \end{pmatrix}$  and  $|w\rangle = \begin{pmatrix} 1 \\ -i \end{pmatrix}$ . Calculate the inner product  $\langle v|w\rangle$  and  $\langle w|v\rangle$ .
- (iv) Which of the following is/are qubits?
- (a)  $\begin{pmatrix} 1 \\ i \end{pmatrix}$  (c)  $\begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{i}{\sqrt{2}} \end{pmatrix}$
- (b)  $|0\rangle + |1\rangle$  (d)  $\sin\theta|+\rangle + |\cos\theta|-\rangle$
- (v) Consider the basis vectors  $|+\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$  and  $|-\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$ . Prove mathematically that they are orthogonal ( $\langle +|-\rangle = 0$ ) and normalized ( $\langle +|+\rangle = 1$ ).
- (vi) Express the standard basis vector  $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$  as a linear combination of  $|+\rangle$  and  $|-\rangle$ .
- (vii) A qubit is in the state  $|\psi\rangle = A(2|0\rangle - i\sqrt{5}|1\rangle)$ . Find the positive real value for  $A$  that makes this a valid quantum state.
- (viii) A qutrit has basis states  $|0\rangle, |1\rangle, |2\rangle$ . Given the state  $|\phi\rangle = \frac{1}{2}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle + c|2\rangle$ , find the value of  $c$  (assuming  $c$  is a positive real number) required to normalize the state.
- (ix) In  $d = 3$ , the computational basis is  $\{|0\rangle, |1\rangle, |2\rangle\}$ . A mutually unbiased basis (similar to the X basis for qubits) is the Fourier basis, where  $|f_k\rangle = \frac{1}{\sqrt{3}} \sum_{j=0}^2 e^{i\frac{2\pi}{3}jk} |j\rangle$  for  $k \in \{0, 1, 2\}$ . Write out the column vector for  $|f_1\rangle$ .
- (x) Given  $|\psi\rangle = \frac{1}{\sqrt{3}}|0\rangle + \sqrt{\frac{2}{3}}e^{i\pi/4}|1\rangle$ . If measured in the computational (Z) basis  $\{|0\rangle, |1\rangle\}$ , what are the probabilities of measuring 0 and 1?
- (xi) If the state  $|\psi\rangle = \frac{1}{\sqrt{3}}|0\rangle + \sqrt{\frac{2}{3}}e^{i\pi/4}|1\rangle$  is measured in the diagonal (X) basis  $\{|+\rangle, |-\rangle\}$ , what is the probability of getting the result corresponding to  $|+\rangle$ ?
- (xii) Alice wants to send the bit sequence 1 0 0 1. She randomly chooses the encoding bases: **Rectilinear (Z), Diagonal (X), Diagonal (X), Rectilinear (Z)**. Write down the sequence of exact quantum states (e.g.,  $|0\rangle, |-\rangle$ , etc.) she transmits to Bob.
- (xiii) Bob receives the four states from above. He randomly chooses to measure them in the following bases: **Diagonal (X), Diagonal (X), Rectilinear (Z), Rectilinear (Z)**.
- (a) For each of the four measurements, state whether Bob's result is 100% deterministic or 50/50 random.
- (b) After Alice and Bob announce their basis choices over a public channel, which bits will they keep to form their raw sifted key?

## CHAPTER 3

### Code-Based Cryptography

In this chapter, we will learn cryptosystems that are based on hard problems from coding theory. It is interesting to note that the first code-based cryptosystem was already proposed in 1978 by Robert J. McEliece and is, thus, as old as the extremely widely used RSA algorithm (and roughly as old as the Diffie-Hellman key exchange, which had been invented one year earlier). One question that immediately comes to mind is why the McEliece cryptosystem has not gained nearly as much popularity as RSA in the past four decades. We will probably address this question. But first we need to provide some background on coding theory, which will be needed to understand code-based cryptosystems. As the name suggests, code-based cryptography is based on error correction codes. They are used for the detection and correction of errors that occur during data transmission, e.g., on a wireless link in a cellular network. Error coding schemes are integral to all modern digital communication, including any type of wired and wireless network connections. We note that error correction codes are also part of every data storage systems, such as computer hard disks or memory sticks. In all these cases (and especially on wireless channels) messages are likely to be corrupted - e.g., through flipping of some bits - so that the other party does not receive the original message correctly.



The basic approach for salvaging the situation is to add some additional, redundant information  $r$  to the original message  $m$ . The message together with the redundant information form what is called a codeword  $c$ . The additional information can simply be attached to the message. In general, the codeword  $c$  does not need to contain the message in clear and  $m$  and  $r$  do not have to be strictly separated from each other, but we restrict ourselves for the sake of clarity to the special case  $c = m|r$  for the rest of this chapter.

If an error is introduced during transmission, an altered codeword  $c'$  will be received. The fundamental assumption is that the receiver can distinguish between valid codewords and corrupted ones. If everything works correctly, the receiver can identify that an error has occurred (in the case of error detection schemes) or even correct the errors  $e$  (in the case of error correction schemes). In the former case, the receiver can request a retransmission of the message by the sender. Not surprisingly, error detection and correction only work if the introduced errors are within the limits that the chosen code can handle, i.e., if too many errors occur, the code cannot successfully detect or correct them.

### 1. Error correcting codes

All communication channels contain some degree of noise, namely interference caused by various sources such as neighboring channels, electric impulses, deterioration of the equipment, etc. This noise can interfere with data transmission. Just as holding a conversation in a noisy room becomes more difficult as the noise becomes louder, so too does data transmission become more difficult as the communication channel becomes noisier. In order to hold a conversation in a loud room, you either raise your voice, or you are forced to repeat yourself. The second method is the one that will concern us; namely, we need to add some redundancy to the transmission in order for the recipient to be able to reconstruct the message. In the following, we give several examples of techniques that can be used. In each case, the symbols in the original message are replaced by codewords that have some redundancy built into them.

EXAMPLE 3.1.1 (repetition codes). Consider an alphabet  $|A, B, C, D$ . We want to send a letter across a noisy channel that has a probability  $p = 0.1$  of error. If we want to send  $C$ , for example, then there is a 90% chance that the symbol received is  $C$ . This leaves too large a chance of error. Instead, we repeat the symbol three times, thus sending  $CCC$ . Suppose an error occurs and the received word is  $CBC$ . We take the symbol that occurs most frequently as the message, namely  $C$ . The probability of the correct message being found is the probability that all three letters are correct plus the probability that exactly one of the three letters is wrong:

$$(0.9)^3 + 3(0.9)^2(0.1) = 0.972,$$

which leaves a significantly smaller chance of error.

Two of the most important concepts for codes are error detection and error correction. If there are at most two errors, this repetition code allows us to detect that errors have occurred. If the received message is  $CBC$ , then there could be either one error from  $CCC$  or two errors from  $BBB$ , we cannot tell which. If at most one error has occurred, then we can correct the error and deduce that the message was  $CCC$ . Note that if we used only two repetitions instead of three, we could detect the existence of one error, but we could not correct it (did  $CB$  come from  $BB$  or  $CC$ ?).

This example was chosen to point out that error correcting codes can use arbitrary sets of symbols. Typically, however, the symbols that are used are mathematical objects such as integers mod a prime

or binary strings. For example, we can replace the letters  $A, B, C, D$  by 2-bit strings: 00, 01, 10, 11. The preceding procedure (repeating three times) then gives us the codewords 000000, 010101, 101010, 111111.

EXAMPLE 3.1.2. (parity check) Suppose we want to send a message of seven bits. Add an eighth bit so that the number of nonzero bits is even. For example, the message 0110010 becomes 01100101, and the message 1100110 becomes 11001100. An error of one bit during transmission is immediately discovered since the message received will have an odd number of nonzero bits. However, it is impossible to tell which bit is incorrect, since an error in any bit could have yielded the odd number of nonzero bits. When an error is detected, the best thing to do is resend the message.

EXAMPLE 3.1.3. (two-dimensional parity code) The parity check code of the previous example can be used to design a code that can correct an error of one bit. The two-dimensional parity code arranges the data into a two-dimensional array, and then parity bits are computed along each row and column. To demonstrate the code, suppose we want to encode the 20 data bits 10011011001100101011. We arrange the bits into a  $4 \times 5$  matrix

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 0 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 | 1 |

and calculate the parity bits along the rows and columns. We define the last bit in the lower right corner of the extended matrix by calculating the parity of the parity bits that were calculated along the columns. This results in the  $5 \times 6$  matrix

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 | 1 |

The parities of the third row and fourth column are odd, so this locates the error as occurring at the third row and fourth column.

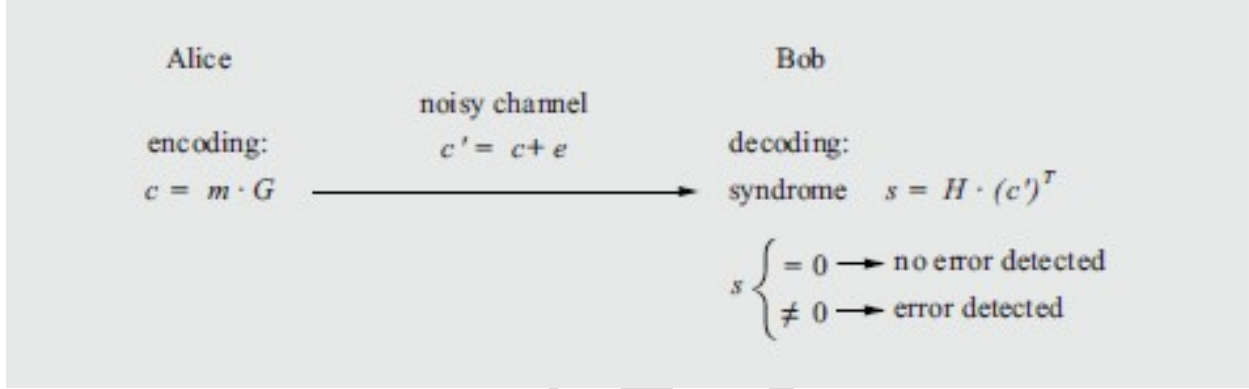
If two errors occur, this code can detect their existence. For example, if bit errors occur at the second and third bits of the second row, then the parity checks of the second and third columns will indicate the existence of two bit errors. However, in this case it is not possible to correct the errors, since there are several possible locations for them. For example, if the second and third bits of the fifth row were incorrect instead, then the parity checks would be the same as when these errors occurred in the second row.

Next, we will introduce a family of linear codes with which we can demonstrate the basic functionality of encoding and decoding.

## 2. Linear Codes

In this section we will introduce the principle of error correction codes. We will use binary linear codes that, as we will see, can easily be decoded. However, for more complex codes, decoding can be a very hard problem that can be used for building asymmetric cryptosystems. This section will be used as a motivation for the syndrome decoding problem, described later, which is at the heart of code-based PQC (post-quantum cryptography). schemes.

The figure shows how data transmission with linear codes works on a high level.

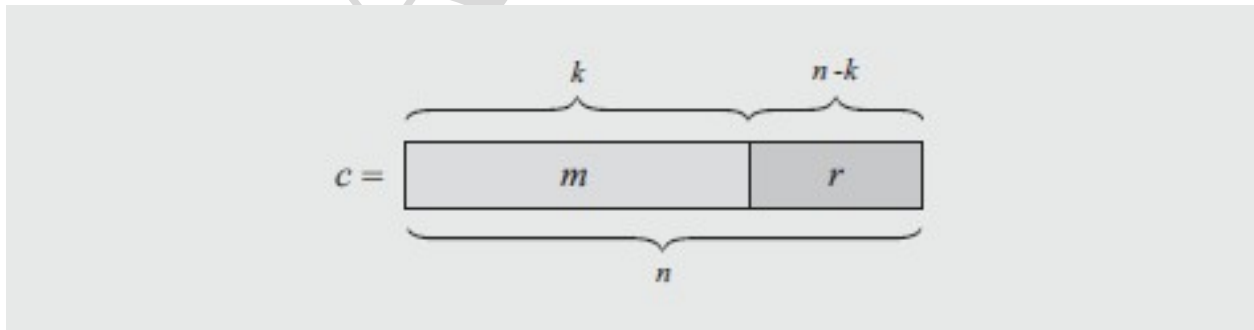


Before we discuss the three stages shown in the figure, let us first define two important parameters, namely  $k$  as the length of the message  $m$ , and  $n$  as the length of the constructed codeword. All lengths are given in bits. From the figure it is clear that the redundant part  $r$  has  $(n - k)$  bits. We now look at the three stages in the error correction process from Figure:

**Encoding:** Alice, the sender, computes a codeword from her message  $m$  by adding some redundant bits. This computation is done by simply multiplying the message with a generator matrix  $G$ :

$$c = mG.$$

As shown in figure, the codeword consists of some redundant information denoted by  $r$  and the message:  $c = m|r$ .



**Transmission:** During data transmission on the channel, errors can occur, resulting in an altered codeword  $c'$ . We consider these errors as bit flips. Mathematically speaking, this can be expressed

by adding an error vector  $e$  to the codeword:

$$c' = c + e.$$

It is important to note that the error is a bit vector so that the addition is a bitwise XOR operation. In other words, at bit locations where the error vector  $e$  has the value “1”, the bit of the codeword  $c$  flips and, thus, an error occurs. At the locations where  $e$  has the “0” bit, no error occurs.

**Decoding:** The receiver Bob has to decode the message. He computes what is referred to as the syndrome  $s$  from the received codeword  $c'$ :

$$s = H(c')^T.$$

The computation is a matrix-vector multiplication with the so-called parity-check matrix  $H$ . The syndrome is crucial for the coding process: If the syndrome  $s$  has the value zero, no error is detected. If  $s$  is not equal to zero, an error has been detected. If not too many bits have flipped, the erroneous bits can even be corrected by Bob.

In the following we will see the mathematical concept behind linear codes, including how the generator matrix  $G$  and the parity-check matrix  $H$  are constructed so that  $r$  depends linearly on  $m$ .

In general, there are two ways to describe a code. The first one is by defining rules to construct all codewords that belong to the code. Exactly this is achieved by the generator matrix  $G$ : Every possible linear combination of the rows of  $G$  corresponds to a valid codeword what we obtain when encoding a message. We multiply  $m$  with  $G$ , which gives us the valid codeword that corresponds to  $m$ .

For the second way, we can define a code based on rules determining whether a given codeword is valid or not. That is the intuition behind the parity-check matrix  $H$ . Whenever we multiply  $H$  with a given word  $\tilde{c}$ , the result is 0 if and only if  $\tilde{c}$  is a valid codeword. This property of the parity-check matrix is fundamental for any error correction code. It allows the receiver to distinguish valid codewords that were constructed by the sender from invalid ones that are corrupted by bit errors. The syndrome gives us this very information due to the following (simple) observation: We recall that the received codeword consists of the original codeword and the error vector, i.e.,  $c' = c + e$ . Hence, the syndrome computation can be expressed as:

$$s = H(c')^T = H(c + e)^T = Hc^T + He^T = 0 + He^T = He^T.$$

Thus, the syndrome  $s$  is always the product of the parity-check matrix and the error vector.

Either of the matrices  $G$  and  $H$  is sufficient to define the code. When knowing the generator matrix  $G$ , one can calculate a corresponding parity-check matrix  $H$  and vice versa. With the following example, we illustrate how to find the generator matrix  $G$  for a given parity-check matrix  $H$  so that we can encode any message.

**EXAMPLE 3.2.1.** Let us assume a message  $m$  consisting of  $k = 4$  bits, so that  $m = (m_1 \ m_2 \ m_3 \ m_4)$ . We want to generate a codeword  $c$  with a length of seven bits ( $n = 7$ ) with three bits ( $n - k = 7 - 4 = 3$ ) of redundancy  $r$ . The codeword has, thus, the form:  $c = (m|r) = (m_1 \ m_2 \ m_3 \ m_4 \ r_1 \ r_2 \ r_3)$ .

The matrix  $H$  has the dimension  $(n - k) \times n = 3 \times 7$ , where the matrix elements are from  $GF(2)$ . We recall that this is the finite field with two elements. We use the following parity-check matrix in this example:

$$Hc^T = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ r_1 \\ r_2 \\ r_3 \end{pmatrix}.$$

We already know the first four bits of the codeword  $c$ , namely  $(m_1 \ m_2 \ m_3 \ m_4)$  but we still need to compute the redundant bits  $(r_1 \ r_2 \ r_3)$ . We recall that an essential property of  $H$  is that  $Hc^T = 0$ . With this, we can derive a system of linear equations:

$$m_2 + m_3 + m_4 + r_1 \equiv 0 \pmod{2}$$

$$m_1 + m_2 + m_4 + r_2 \equiv 0 \pmod{2}$$

$$m_1 + m_3 + m_4 + r_3 \equiv 0 \pmod{2}$$

This linear equation set can be rewritten as:

$$r_1 \equiv m_2 + m_3 + m_4 \pmod{2}$$

$$r_2 \equiv m_1 + m_2 + m_4 \pmod{2}$$

$$r_3 \equiv m_1 + m_3 + m_4 \pmod{2}$$

The three equations determine the redundant part  $r$  from the 4-bit message  $m$  so that we can generate a valid codeword  $c$ . Since the three bits  $r_i$  are linear combinations of the message bits, we can write the construction of the codeword as a matrix multiplication (which is also a linear operation):

$$c = \begin{pmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ r_1 \\ r_2 \\ r_3 \end{pmatrix}^T = \begin{pmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ m_2 + m_3 + m_4 \\ m_1 + m_2 + m_4 \\ m_1 + m_3 + m_4 \end{pmatrix}^T = \underbrace{(m_1 \ m_2 \ m_3 \ m_4)}_m \underbrace{\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}}_G = mG.$$

This example showed how a generator matrix  $G$  can be constructed from  $H$ . It has size  $4 \times 7$ , i.e., it consists of  $k = 4$  rows and  $n = 7$  columns.

**DEFINITION 3.2.2 ((Binary) Linear Codes).** Given a parity-check matrix  $H$  of size  $(n - k) \times n$  over any prime field  $GF(p)$ . For binary linear codes, the field is  $GF(2)$ . We define the linear code  $C$  as the set of all vectors  $c \in GF(2)^n$  with  $n$  elements for which  $Hc^T = 0$  holds.

The set  $C$  defines all valid codewords of a linear  $(n, k)$  code, where  $n$  is called the length and  $k$  the dimension of the code.

Any  $k \times n$  matrix  $G$  whose row space is equal to  $C$  is called a generator matrix of  $C$  and can be used for encoding by  $c = mG$ . A code can equivalently be defined by its generator matrix instead of its parity-check matrix.

The row space used in the definition above is simply the set of all possible linear combinations of the rows of  $H$ .

With this definition, there are multiple options for constructing a linear code. Simple ones are known as parity-check or repetition codes. We focus on simple (systematic) binary linear codes for clarity, while more complex linear codes over other finite fields exist which are also widely used in practice. We will now look at another example that allows us to discover some fundamental aspects of error correction codes, in particular the notion of error correction capability.

**EXAMPLE 3.2.3.** We again assume a linear code with the parameters  $k = 4$ ,  $n = 7$ . The generator matrix is the same as in the previous example:

$$G = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}.$$

The corresponding parity-check matrix with  $n - k = 3$  and  $n = 7$  is defined by:

$$H = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{pmatrix}.$$

A key feature of codes is their maximum correction capability  $t$ . This parameter denotes the maximum number of bit flips during transmission that can be corrected. In order to derive  $t$ , we have to look at all valid codewords. We then have to determine the minimum Hamming distance between all those codewords. The Hamming distance between two bit vectors is the number of positions at which they differ. For instance, the Hamming distance between  $(1 \ 1 \ 0 \ 1)$  and  $(1 \ 0 \ 1 \ 1)$  is two since they differ in the second and third bit position.

In the code we consider here, we have  $k = 4$  and, thus, there are  $2^4 = 16$  possible messages, which lead to 16 valid codewords. As an example, we consider the message  $m = (1 \ 0 \ 1 \ 0)$ . As shown

earlier, the encoding of the message is obtained by multiplying  $m$  with the generator matrix  $G$ , i.e.,

$$c = mG = (1\ 0\ 1\ 0) \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix} = ..(1\ 0\ 1\ 0\ 1\ 1\ 0).$$

If we do the same with all possible four-bit messages, we obtain an exhaustive list with all possible messages  $m$  and their corresponding codewords  $c$  as presented below.

| Messages $m$ | Codewords $c$   |
|--------------|-----------------|
| (0 0 0 0)    | (0 0 0 0 0 0 0) |
| (0 0 0 1)    | (0 0 0 1 1 1 1) |
| (0 0 1 0)    | (0 0 1 0 1 0 1) |
| (0 0 1 1)    | (0 0 1 1 0 1 0) |
| (0 1 0 0)    | (0 1 0 0 1 1 0) |
| (0 1 0 1)    | (0 1 0 1 0 0 1) |
| (0 1 1 0)    | (0 1 1 0 0 1 1) |
| (0 1 1 1)    | (0 1 1 1 1 0 0) |
| (1 0 0 0)    | (1 0 0 0 0 1 1) |
| (1 0 0 1)    | (1 0 0 1 1 0 0) |
| (1 0 1 0)    | (1 0 1 0 1 1 0) |
| (1 0 1 1)    | (1 0 1 1 0 0 1) |
| (1 1 0 0)    | (1 1 0 0 1 0 1) |
| (1 1 0 1)    | (1 1 0 1 0 1 0) |
| (1 1 1 0)    | (1 1 1 0 0 0 0) |
| (1 1 1 1)    | (1 1 1 1 1 1 1) |

We now have to determine the Hamming distance between all possible pairs of codewords. If we do that, we find that the minimum Hamming distance is  $d = 3$ . This means that any two codewords differ in at least three bit positions. From here it is easy to see that the error correction capability is

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor = 1.$$

Behind this expression is the principle that if  $t = 1$  bit errors have been introduced, there is a nearest code word, which contains the original message. This nearest code word  $c$  has a Hamming distance of one from the received  $c'$ , while all other codewords will have a Hamming distance of at least two. In other words, the given linear code is capable of correcting any single-bit error that may occur during transmission. Such an error will flip exactly one bit in any of the codewords listed above. There are seven error vectors  $e = (e_1, \dots, e_7)$  that will cause such a bit flip: The error vectors have one bit  $e_i$  with the value 1 and all other bit values are 0.

We can now use the seven error vectors to compute all syndromes that allow a correction of a one-bit error. We achieve this by multiplying these error vectors with the parity-check matrix. We recall that this is due to the relationship  $s = H(c')^T = He^T$ . As an example, we consider the error vector  $e = (0\ 0\ 0\ 0\ 1\ 0\ 0)$ . This vector leads to the following syndrome  $s$ :

$$s^T = He^T = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}.$$

If we repeat this computation with all seven error vectors we obtain the list below:

| Error $e$               | Syndrome $s$ |
|-------------------------|--------------|
| $(0\ 0\ 0\ 0\ 0\ 0\ 1)$ | $(0\ 0\ 1)$  |
| $(0\ 0\ 0\ 0\ 0\ 1\ 0)$ | $(0\ 1\ 0)$  |
| $(0\ 0\ 0\ 0\ 1\ 0\ 0)$ | $(1\ 0\ 0)$  |
| $(0\ 0\ 0\ 1\ 0\ 0\ 0)$ | $(1\ 1\ 1)$  |
| $(0\ 0\ 1\ 0\ 0\ 0\ 0)$ | $(1\ 0\ 1)$  |
| $(0\ 1\ 0\ 0\ 0\ 0\ 0)$ | $(1\ 1\ 0)$  |
| $(1\ 0\ 0\ 0\ 0\ 0\ 0)$ | $(0\ 1\ 1)$  |

The table is essential for decoding: If the receiver, Bob, computes any of the seven syndromes show in the right column, he can look up the corresponding error vector. To correct the error, he simply adds  $e$  to the received codeword  $c'$  and obtains the codeword that was originally sent by Alice:  $c = c' + e$ . This immediately gives Bob the original message since the first  $k$  bits of the code word are the message.

### 3. The Syndrome Decoding Problem

As we know, all asymmetric cryptosystems are based on a hard problem that an attacker cannot solve. To build a PQC scheme based on codes, our goal is to turn the decoding step into a computationally hard problem. Unfortunately, decoding in the example above seems straightforward and not particularly difficult, as it only requires simple matrix operations. However, the ease with which we could decode (and thus find the error) was due to the fact that we used a small code for which the error correction capability was very low, namely  $t = 1$ . If we increase  $t$ , the decoding step becomes significantly more complex in the general case. To get a feeling for that, let us assume an error correction capability of up to  $t = 8$  for a code with  $n = 2048$  bits. While the size of this code seems already large, we will see later that we need even larger values for  $n$  to achieve secure code-based cryptosystems.

If we compute the table with all possible syndromes for such a code as done in the example above, the table becomes very large since we need to consider all error combinations over the range from 1 to 8 possible errors. The number of error combinations is given by the corresponding binomial coefficients:

$$\sum_{i=1}^8 \binom{2048}{i} \approx 2^{65}.$$

Such a lookup table would be absurdly large, as it would contain 32 million terabytes of entries. In fact, unless there is mathematical structure in the code that allows a computationally easy way to map the syndrome to the error vector  $e$  with a Hamming weight smaller than or equal to  $t$ , this is a hard problem in the general case. It is known as the syndrome decoding problem and was identified by Berlekamp, McEliece and van Tilborg in 1978. We give now a formal definition of the problem.

**DEFINITION 3.3.1.** If  $m$  is a finite binary string, then the Hamming weight,  $HW(m)$ , of  $m$  is defined to be the number one 1s in the string.

For example,  $HW(00011101) = 4$ ,  $HW(010101111) = 6$ .

**DEFINITION 3.3.2 (Syndrome Decoding Problem).** Given a (linear) code with parity-check matrix  $H$  of size  $(n - k) \times n$  over the binary field  $GF(2)$ , a syndrome  $s$  consisting of  $n - k$  bits and the maximum error correcting capability  $t$ . We call the task to find an  $n$ -bit vector  $e$  that does not exceed the Hamming weight  $HW(e) \leq t$  such that  $He^T = s$  the syndrome decoding problem.

The constraint  $HW(e) \leq t$  makes the problem a very difficult one in the general case. In fact, it has an exponential run time in the worst case. Even though the syndrome decoding problem is promising for cryptography, it is not useful for error correction codes, where the goal is to find the error vector that corrupted the data. For this reason, since the 1960s, the coding community has developed a large number of error correction codes that allow an efficient decoding process. The basic principle is always to introduce mathematical structure to the codes so that decoding becomes feasible. Well-known decoding algorithms for error correction codes used in practice are, e.g., the Berlekamp-Massey and the Patterson algorithms.

In order to build public-key schemes one has to combine two conflicting goals: An adversary should face the (computationally very difficult) syndrome decoding problem, whereas the owner of the private key should be able to use an efficient decoding algorithm. The idea of code-based PQC schemes is that Bob, the owner of the private/public key pair, uses a code for which he has an efficient decoding algorithm. However, he does not make the original code public but hides the code by adding randomness to it. For instance, in the McEliece scheme, the original code that Bob can decode is given by the generator matrix  $G$ . If an attacker knew  $G$ , he could also decode. Therefore Bob only publishes  $\hat{G}$  in which he hides  $G$  by multiplying it with two random matrices:

$$k_{\text{pub}} = \hat{G} = S \times G \times P.$$

Below we will introduce details of two asymmetric encryption schemes that are based on this principle.

#### 4. Encryption Schemes

Code-based PQC schemes can be used for encrypting of data, and are used in practice particularly for key transport. In a nutshell, Bob publishes a hidden linear code within his public key. If Alice wants to send a plaintext message, she performs an encoding operation of that linear code. She also artificially introduces an error during the encoding operation. When Bob receives the encoded message, he runs the decoding operation, which removes the errors, so that he obtains the plaintext. However, decoding is only easy and efficient if the actual code is known. An attacker, Oscar, who eavesdrops, can therefore not retrieve the message correctly since Bob hides the code before publishing it. The security of these code-based schemes is based on the assumption that the adversary is not able to solve the syndrome decoding problem or a related problem for the (seemingly) random code in the public key. We now introduce the first code-based cryptosystem, which was proposed by Robert McEliece in 1978, and subsequently the Niederreiter encryption scheme.

**4.1. The McEliece Cryptosystem.** To build a public-key encryption system, McEliece observed that we can easily transform the encoding operation of an error correction code into encryption and, likewise, decoding into a decryption operation. In order to make this approach work securely, one needs some additional operations that hide the underlying code structure but still allow the syndrome decoding if one knows the private key. The idea for hiding the code structure is simple: McEliece proposed to perturb the generator matrix  $G$  by multiplying it with two randomly chosen matrices: a scrambling matrix  $S$  and permutation matrix  $P$ . Bob - as legitimate user - keeps the unscrambled code as his private key and can, thus, efficiently decrypt using a standard decoding algorithm. An attacker, however, without access to the unscrambled underlying code is left to solve the hard problem of decoding a general linear code. Below, we introduce the three steps of the McEliece scheme, key generation, encryption and decryption.

##### McEliece Key Generation

**Output:** public key:  $k_{pub} = \hat{G}$  and private key  $k_{pr} = (S, G, P)$ .

Given are a linear code  $C[n, k, d]$  with maximum error correcting capability  $t$ , a generator matrix  $G$  and corresponding decoding function  $\text{decode}(\cdot)$ .

- (i) Generate the private key  $k_{pr} = (S, G, P)$ , where  $S$  is a  $k \times k$  random, invertible matrix and  $P$  an  $n \times n$  random, invertible permutation matrix.
- (ii) Compute the public key  $k_{pub} = \hat{G} = S \times G \times P$ .

##### McEliece Encryption

- (i) Generate a random  $n$ -bit error vector  $e$  of weight  $t$ .
- (ii) Given a message  $m$  and  $k_{pub}$ , compute the ciphertext:  $c = m\hat{G} + e$ .

##### McEliece Decryption

- (i) Compute  $u = cP^{-1}$ .
- (ii) Decode the intermediate value  $v = \text{decode}(u)$ .
- (iii) Compute  $m = vS^{-1}$ .

EXAMPLE 3.4.1. We assume that Bob computes a public/private key pair, Alice encrypts a message and sends it to Bob, who eventually decrypts the ciphertext using his private key. Bob uses the linear code as discussed earlier, i.e., he has the generator matrix  $G$  and parity-check matrix  $H$ .

We start with the key generation. For the construction of the private key  $k_{pr} = (S, G, P)$ , Bob generates random matrices  $S$  and  $P$ . He needs to ensure that  $S$  is invertible. Since  $P$  is a permutation matrix, it is invertible by definition. In this example, we assume that  $S$  and  $P$  are given by:

$$S = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \text{ and } P = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}.$$

We note that  $P$  is in fact a permutation matrix and it can be shown that  $S$  is in fact invertible. Their inverses are

$$S^{-1} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 \end{pmatrix} \text{ and } P^{-1} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

Given  $S$  and  $P$ , Bob can now compute the public key  $k_{pub} = \hat{G}$ , where all arithmetic is done in  $\mathbb{Z}_2$ .

$$\hat{G} = SGP = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

$$= \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 \end{pmatrix}.$$

When Alice wants to encrypt the message  $m = (m_1 \ m_2 \ m_3 \ m_4)$ , she first generates random bits for the error vector  $e = (e_1 \ e_2 \ \dots \ e_7)$ . She then encodes  $m$  with Bob's public-key matrix  $\hat{G}$ , adding the error  $e$ . Since the code in this example can correct  $t = 1$  error, Alice must choose an error vector with a Hamming weight of one.

We assume Alice's error vector  $e$  is:

$$e = (0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0).$$

The message Alice wants to encrypt is  $m = (1 \ 0 \ 1 \ 1)$ .

For the actual encryption, Alice computes the ciphertext  $c$  by multiplying  $m$  with  $k_{pub}$  and adding the error vector  $e$ .

$$\begin{aligned} c &= m\hat{G} + e = (1 \ 0 \ 1 \ 1) \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 \end{pmatrix} + (0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0) \\ &= (0 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0). \end{aligned}$$

This ciphertext  $c$  is sent to Bob. Upon reception, he can decrypt using the known code and the private-key matrices  $P^{-1}$  and  $S^{-1}$ .

First, Bob computes the intermediate value  $u$  using the ciphertext and the inverse of  $P$ :

$$u = cP^{-1} = (0 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0) \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{pmatrix} = (0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1).$$

Next, Bob has to decode  $u$ . For this, he uses the decoding algorithm that is specific to the chosen underlying code. Decoding means that he removes the errors that were artificially introduced by Alice. Therefore, he first multiplies the parity-check matrix  $H$  by the vector  $u^T$ , i.e.,

$$s' = Hu^T = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{pmatrix} (0 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1)^T = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}.$$

With this syndrome  $s'$ , Bob can determine the permuted error  $e' = eP^{-1}$ . Using the look-up table we prepared earlier, he identifies the permuted error vector as

$$e' = (0\ 0\ 0\ 0\ 0\ 1\ 0).$$

To undo the permutation of the bit positions of  $e'$  we find  $e$  by multiplying  $e'$  with the permutation matrix  $P$ , i.e.,  $e = e'P$ .

Next, Bob uses the permuted error vector  $e'$  to correct the intermediate vector  $u$ . Since all of our computations are accomplished in  $\mathbb{Z}_2$ , we can remove  $e'$  from  $u$  by a simple addition modulus two. Bob obtains the corrected vector  $\tilde{u}$  by

$$\tilde{u} = u + e' = (0\ 0\ 0\ 1\ 1\ 0\ 1) + (0\ 0\ 0\ 0\ 0\ 1\ 0) = (0\ 0\ 0\ 1\ 1\ 1\ 1).$$

Since Bob is only interested in the message, we remove the last  $n - k = 3$  bits from  $\tilde{u}$  to obtain  $v$  as the result of the decoding process:

$$v = \text{decode}(u) = (0\ 0\ 0\ 1).$$

Finally, Bob multiplies  $v$  by the inverse of  $S$  and recovers the original message  $m$ :

$$m = vS^{-1} = (0\ 0\ 0\ 1) \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 \end{pmatrix} = (1\ 0\ 1\ 1).$$